

Операционные системы

Лекция 15

**Функции операционной системы по
управлению файлами и устройствами**

План лекции

- Задачи ОС по управлению файлами и устройствами
- Многослойная модель подсистемы ввода-вывода: общая схема, менеджер ввода-вывода, многоуровневые драйверы, специальные файлы
- Логическая организация файловой системы: цели и задачи файловой системы, типы файлов, иерархическая структура файловой системы, имена файлов, монтирование, атрибуты файлов
- Логическая организация файла
- Средства перечисления элементов файловой системы

Ввод-вывод и файловая система

Одной из главных задач ОС является обеспечение обмена данными между приложениями и периферийными устройствами компьютера. Собственно, ради выполнения этой задачи и были разработаны первые системные программы, послужившие прототипами операционных систем

В современной ОС функции обмена данными с периферийными устройствами выполняет подсистема ввода-вывода

Клиентами этой подсистемы являются не только пользователи и приложения, но и некоторые компоненты самой ОС, которым требуется получение системных данных или их вывод, например, подсистеме управления процессами при смене активного процесса необходимо записать на диск контекст приостанавливаемого процесса и считать с диска контекст активизируемого процесса

Ввод-вывод и файловая система

Основными компонентами подсистемы ввода-вывода являются **драйверы**, управляющие внешними устройствами, и **файловая система**

К подсистеме ввода-вывода можно также с некоторой долей условности отнести рассмотренный ранее диспетчер прерываний. Условность заключается в том, что диспетчер прерываний обслуживает не только модули подсистемы ввода-вывода, но и другие модули ОС, в частности такой важный модуль, как планировщик/диспетчер потоков

Но из-за того, что планирование работ подсистемы ввода-вывода составляет основную долю нагрузки диспетчера прерываний, его вполне логично рассматривать как ее составную часть (к тому же первопричиной появления в компьютерах системы прерываний были в свое время именно операции с устройствами ввода-вывода)

Задачи ОС по управлению файлами и устройствами

Подсистема ввода-вывода (Input-Output Subsystem) мультипрограммной ОС при обмене данными с внешними устройствами компьютера должна решать ряд общих задач, из которых наиболее важными являются следующие:

- Организация параллельной работы устройств ввода-вывода и процессора
- Согласование скоростей обмена и кэширование данных
- Разделение устройств и данных между процессами
- Предоставление удобного программного интерфейса к устройствам
- Поддержка широкого спектра драйверов с возможностью простого включения в систему нового драйвера
- Динамическая загрузка и выгрузка драйверов
- Поддержка нескольких файловых систем
- Поддержка синхронных и асинхронных операций ввода-вывода

Параллельная работа I/O-устройства и CPU

Параллельная работа устройства ввода-вывода и процессора организуется путем взаимодействия **контроллера** и **драйвера**, относящихся к данному устройству

Контроллер – это специализированный блок управления, которым снабжается каждое устройство ввода-вывода вычислительной системы, включая диск, принтер, терминал и т. п.

Драйвер – программный модуль ОС, предназначенный для управления устройством ввода-вывода

Параллельная работа I/O-устройства и CPU

Контроллер периодически принимает от драйвера выводимую на устройство информацию, а также команды управления, которые говорят о том, что с этой информацией нужно сделать (например, вывести в виде текста в определенную область экрана или записать в определенный сектор диска)

Под управлением контроллера устройство может некоторое время выполнять свои операции автономно, не требуя внимания со стороны центрального процессора, а следовательно, и со стороны ОС. Это время зависит от многих факторов – объема выводимой информации, степени интеллектуальности управляющего устройством контроллера, быстродействия устройства и т. п.

Параллельная работа I/O-устройства и CPU

Даже самый примитивный контроллер, выполняющий простые функции, обычно тратит довольно много времени на самостоятельную реализацию подобной функции после получения очередной команды от процессора

Это же справедливо и для сложных контроллеров, так как скорость работы любого устройства ввода-вывода, даже самого быстродействующего, обычно существенно ниже скорости работы процессора

Параллельная работа I/O-устройства и CPU

Итак, в периоды между выдачами команд от процессора контроллеры выполняют свои действия независимо от ОС

От подсистемы ввода-вывода ОС требуется «лишь» осуществлять запуск и приостановку драйверов всех устройств ввода-вывода, входящих в состав компьютера, обеспечивая приемлемое время реакции каждого драйвера на сигналы прерываний, поступающие от соответствующих контроллеров

При этом время, которое тратит процессор на обеспечение ввода-вывода, должно быть минимизировано в пользу пользовательских процессов

Параллельная работа I/O-устройства и CPU

Данная задача является классической задачей планирования/диспетчеризации процессов в системах реального времени и обычно решается на основе многоуровневой приоритетной схемы обслуживания по прерываниям

Для обеспечения приемлемого уровня реакции все драйверы (или части драйверов) распределяются по нескольким приоритетным уровням, в соответствии с временем реакции, допустимым для данного типа устройства, и временем, затрачиваемым процессором на выполнение соответствующего драйвера

Для реализации приоритетной схемы обычно задействуется общий диспетчер прерываний ОС

Согласование скоростей обмена и кэширование данных

При обмене данными всегда возникает задача согласования скорости

Например, если один пользовательский процесс вырабатывает некоторые данные и передает их другому пользовательскому процессу через оперативную память, то в общем случае скорости генерации данных и их чтения не совпадают

Согласование скорости при обмене данными между *прикладными процессами* обычно достигается за счет **буферизации данных в оперативной памяти** и синхронизации доступа процессов к буферу

Согласование скоростей обмена и кэширование данных

Аналогичным образом решается проблема согласования скоростей обмена в подсистеме ввода-вывода

Здесь также широко используется буферизация данных в оперативной памяти

В тех специализированных операционных системах, в которых обеспечение высокой скорости ввода-вывода является первоочередной задачей (управление в реальном времени, услуги сетевой файловой службы и т. п.), большая часть оперативной памяти отводится не под коды прикладных программ, а под буферы данных

Согласование скоростей обмена и кэширование данных

Однако буферизация только на основе оперативной памяти в подсистеме ввода-вывода оказывается недостаточной – разница между скоростью обмена с оперативной памятью, куда процессы помещают данные для обработки, и скоростью работы внешнего устройства часто становится слишком значительной, чтобы в качестве временного буфера можно было бы использовать оперативную память – ее объема может просто не хватить

В системах ввода-вывода при необходимости обмена большими объемами данных используется **спулинг**, при котором данные буферизуются на диске в особой области, называемой **дисковым файлом**, или **спул-файлом**

Согласование скоростей обмена и кэширование данных

Типичный пример применения спул-файла дает организация вывода данных на принтер

Для печатаемых документов объем в несколько десятков мегабайтов – не редкость, поэтому для их временного хранения (а печать каждого документа занимает от нескольких минут до десятков минут) объема оперативной памяти явно недостаточно (речь конечно же об оперативной памяти принтера)

Согласование скоростей обмена и кэширование данных

Другим решением этой проблемы является использование большой буферной памяти в контроллерах внешних устройств

Такой подход особенно полезен в тех случаях, когда помещение данных на диск слишком замедляет обмен (или когда данные выводятся на сам диск)

Например, в контроллерах графических дисплеев применяется буферная память, соизмеримая по объему с оперативной, и это существенно ускоряет вывод графики на экран

Согласование скоростей обмена и кэширование данных

Буферизация данных при вводе-выводе позволяет не только согласовать скорости работы процессора и внешнего устройства, но и решить другую задачу – сократить количество реальных операций ввода-вывода за счет кэширования данных

Дисковый кэш является неизменным атрибутом подсистем ввода-вывода практически всех операционных систем, значительно сокращая время доступа к хранимым данным

Разделение устройств и данных

Устройства ввода-вывода могут предоставляться процессам как в **монопольное**, так и в **совместное** (разделяемое) использование

Какой тип доступа к устройству разрешен тому или иному процессу, определяет операционная система

Контроль доступа осуществляется теми же способами, что и при доступе процессов к другим ресурсам вычислительной системы – путем проверки прав пользователя или группы пользователей, от имени которых действует процесс, на выполнение той или иной операции с устройством

Например, определенной группе пользователей может быть разрешено захватывать последовательный порт в монопольное владение, в то время как другим пользователям – запрещено

Разделение устройств и данных

Одно и то же устройство в разные периоды времени может использоваться как в разделяемом, так и в монопольном режимах

Тем не менее существуют устройства, для которых обычно характерен один из этих режимов, например, последовательные порты и алфавитно-цифровые терминалы чаще используются в монопольном режиме, а диски – в режиме совместного доступа

Операционная система должна предоставлять эти устройства в обоих режимах: в монопольном режиме отслеживая процедуры захвата и освобождения устройств, а в режиме разделения оптимизируя последовательность операций ввода-вывода для различных процессов в целях повышения общей производительности, если это возможно

Разделение устройств и данных

Например, при обмене данными нескольких процессов с диском можно так упорядочить последовательность операций, что непроизводительные затраты времени на перемещение головок существенно сократятся (при этом для отдельных процессов возможно некоторое замедление операций ввода-вывода)

Операционная система может контролировать доступ не только к устройству в целом, но и к отдельным **порциям данных**, хранимых или отображаемых этим устройством

Разделение устройств и данных

Диск является типичным примером устройства, для которого важно контролировать доступ не к устройству в целом, а к отдельным каталогам и файлам

При выводе информации на графический дисплей отдельные окна на экране также представляют собой ресурсы, к которым необходимо обеспечить тот или иной вид доступа для протекающих в системе процессов

При этом для каждой порции данных или части устройства могут быть заданы свои права доступа, не связанные напрямую с правами доступа к устройству в целом

Так, в файловой системе для каждого каталога и файла обычно можно задать индивидуальные права доступа. Очевидно, что для организации совместного доступа к частям устройства или данным, хранящихся на нем, неременным условием является задание режима совместного использования устройства в целом

Разделение устройств и данных

При разделении устройства несколькими процессами может возникнуть необходимость в **разграничении** порций данных, относящихся к каждому из этих процессов

Такая потребность возникает при совместном использовании так называемых последовательных устройств, данные в которых, в отличие от устройств прямого доступа, не адресуются

Типичным примером такого рода устройства является принтер, который, с одной стороны, не выделяется в монопольное владение процессам, так как это устройство достаточно дорогое, а с другой – его совместное использование может вызвать проблему

Например, пусть две программы, работая в мультизадачном режиме, выводят на печать документы. Каждая программа выдает данные на печать порциями в своем темпе

Разделение устройств и данных

Без принятия специальных мер на выходе принтера была бы получена мешанина строк, относящихся к двум разным документам, созданным каждой из этих двух программ

Поэтому подсистема ввода-вывода для подобных устройств организует очередь так называемых **заданий на вывод**, при этом с каждым заданием связана порция данных, которую нельзя разрывать

Для хранения очереди заданий используется спул-файл, который одновременно согласует скорости работы принтера и оперативной памяти и позволяет организовать разбиение данных на логические порции

Так как спул-файл находится на разделяемом устройстве прямого доступа, то процессы могут одновременно выполнять вывод на принтер, помещая данные в свой раздел спул-файла

Программный интерфейс к устройствам

Для того чтобы облегчить разработку прикладных программ, включающих процедуры ввода-вывода, ОС должна поддерживать экранирующий логический интерфейс между периферийными устройствами и приложениями» который давал бы возможность использовать общий набор базовых операций ввода-вывода для любых устройств независимо от их специфики

В качестве основы такого интерфейса практически все современные операционные системы поддерживают **файловую модель устройств ввода-вывода**

Здесь мы еще раз сталкиваемся с плодотворной концепцией виртуализации

Программный интерфейс к устройствам

Все разнообразие типов реальных устройств ввода-вывода операционная система подменяет одним виртуальным типом устройства

Все виртуальные устройства работают единым образом и представляются в виде файлов, называемых также **специальными**, или **виртуальными**, файлами

Каждому устройству ввода-вывода ставится в соответствие отдельный специальный файл. Он представляет это устройство для прикладных процессов и остальной части операционной системы в виде неструктурированного набора байтов

В результате вместо написания сложных процедур ввода-вывода для реальных устройств программист может теперь просто использовать операции чтения из специальных файлов и записи в специальные файлы, ассоциированные с этими устройствами

Программный интерфейс к устройствам

Привлекательность модели файла-устройства состоит в ее простоте и универсальности для устройств любого типа.

Однако во многих случаях, как, например, вывода графической информации на дисплей или принтер, программирования операций сетевого обмена и др., эта модель является слишком примитивной и не освобождает прикладного программиста от рутинных задач программирования ввода-вывода для этих устройств

Поэтому для некоторых типов устройств ОС поддерживает более развитый интерфейс, отражающий их специфику

Поддержка широкого спектра драйверов

Достоинством подсистемы ввода-вывода любой универсальной ОС является наличие разнообразного набора драйверов для наиболее популярных периферийных устройств

Прекрасно спланированная и реализованная операционная система может потерпеть неудачу на рынке только из-за того, что в ее состав не включен достаточный набор драйверов, в результате администраторы и пользователи вынуждены будут искать нужный им драйвер для имеющегося у них внешнего устройства у производителей оборудования или, что еще хуже, заниматься его разработкой

Именно в такой ситуации оказались пользователи первых версий OS/2, и, возможно, данное обстоятельство послужило в свое время не последней причиной того, что эта неплохая операционная система сдала позиции богатой на драйверы ОС Windows 3.x.

Поддержка широкого спектра драйверов

Чтобы операционная система не испытывала недостатка в драйверах, необходимо наличие логически ясного, удобного и открытого интерфейса между драйверами и другими компонентами ОС

Такой интерфейс нужен для того, чтобы драйверы писали не только непосредственные разработчики данной операционной системы, но и большая армия программистов по всему миру, в первую очередь – тех предприятий, которые выпускают внешние устройства для компьютеров

Открытость интерфейса драйверов, то есть доступность его описания для независимых разработчиков программного обеспечения (а, возможно, также и разработка его на основе согласительных процедур между ведущими коллективами разработчиков), является необходимым условием успешного развития операционной системы

Поддержка широкого спектра драйверов

Драйвер должен поддерживать два типа интерфейсов:

- **интерфейс «драйвер-ядро»** (Driver Kernel Interface, **DKI**) с модулями ядра ОС (модулями подсистемы ввода-вывода, системных вызовов, подсистем управления процессами и памятью и т. д.)
- **интерфейс «драйвер-устройство»** (Driver Device Interface, **DDI**) с контроллерами внешних устройств

Интерфейс «драйвер-ядро» должен быть стандартизован в любом случае, а интерфейс «драйвер-устройство» имеет смысл стандартизировать тогда, когда подсистема ввода-вывода не разрешает драйверу непосредственно взаимодействовать с аппаратурой контроллера, а выполняет эти операции самостоятельно

Поддержка широкого спектра драйверов

Экранирование драйвера от аппаратуры является весьма полезной функцией, так как драйвер в этом случае становится независимым от аппаратной платформы.

Подсистема ввода-вывода может поддерживать несколько различных типов интерфейсов DKI/DDI, предоставляя специфический интерфейс для устройств определенного класса

Так, в ОС семейства Windows NT для драйверов сетевых адаптеров предусмотрен интерфейс стандарта NDIS (Network Driver Interface Specification), в то время как драйверы сетевых транспортных протоколов взаимодействуют с верхними слоями сетевого программного обеспечения по интерфейсу TDI (Transport Driver Interface)

Для поддержки процесса разработки драйверов операционной системы обычно выпускается так называемый пакет **DDK (Driver Development Kit)**, представляющий собой набор соответствующих инструментальных средств – библиотек, компиляторов и отладчиков

Динамическая загрузка и выгрузка драйверов

Набор потенциально поддерживаемых данной ОС периферийных устройств всегда существенно шире набора устройств, которыми ОС должна управлять при установке на конкретной машине

Поэтому ценным свойством ОС является способность **динамически** загружать в оперативную память требуемый драйвер (без останова ОС) и выгружать его после того, как потребность в поддержке устройства миновала, что может существенно сэкономить системную область памяти

Поддержка динамической загрузки драйверов является практически обязательным требованием для современных универсальных операционных систем

Динамическая загрузка и выгрузка драйверов

Альтернативой динамической загрузке драйверов при изменении текущей конфигурации внешних устройств компьютера является повторная компиляция кода ядра с требуемым набором драйверов, что создает между всеми компонентами ядра статические связи вместо динамических

Например, таким образом решалась данная проблема в ранних версиях операционной системы Unix

При статических связях между ядром и драйверами структура ОС упрощается, но этот подход требует наличия исходных кодов модулей операционной системы, доступность которых скорее является исключением (для некоммерческих версий Unix/Linux), а не правилом

Кроме того, в этом варианте предыдущую работающую версию операционной системы необходимо остановить и заменить новой, а перерывы в работе ОС в некоторых применениях могут и не допускаться

Поддержка файловых систем

Диски представляют собой особый род периферийных устройств, так как именно на них хранится большая часть как пользовательских, так и системных данных

Данные на дисках организуются в файловые системы, и свойства файловой системы во многом определяют свойства самой ОС

Популярность файловой системы часто приводит к ее миграции из «родной» ОС в другие операционные системы, например, файловая система FAT появилась первоначально в MS-DOS, но затем была реализована в OS/2, семействе MS Windows и многих реализациях Unix

Поддержка файловых систем

Ввиду этого, поддержка нескольких популярных файловых систем для подсистемы ввода-вывода так же важна, как и поддержка широкого спектра периферийных устройств

Важно также, чтобы архитектура подсистемы ввода-вывода позволяла достаточно просто включать в ее состав новые типы файловых систем без необходимости переписывания кода

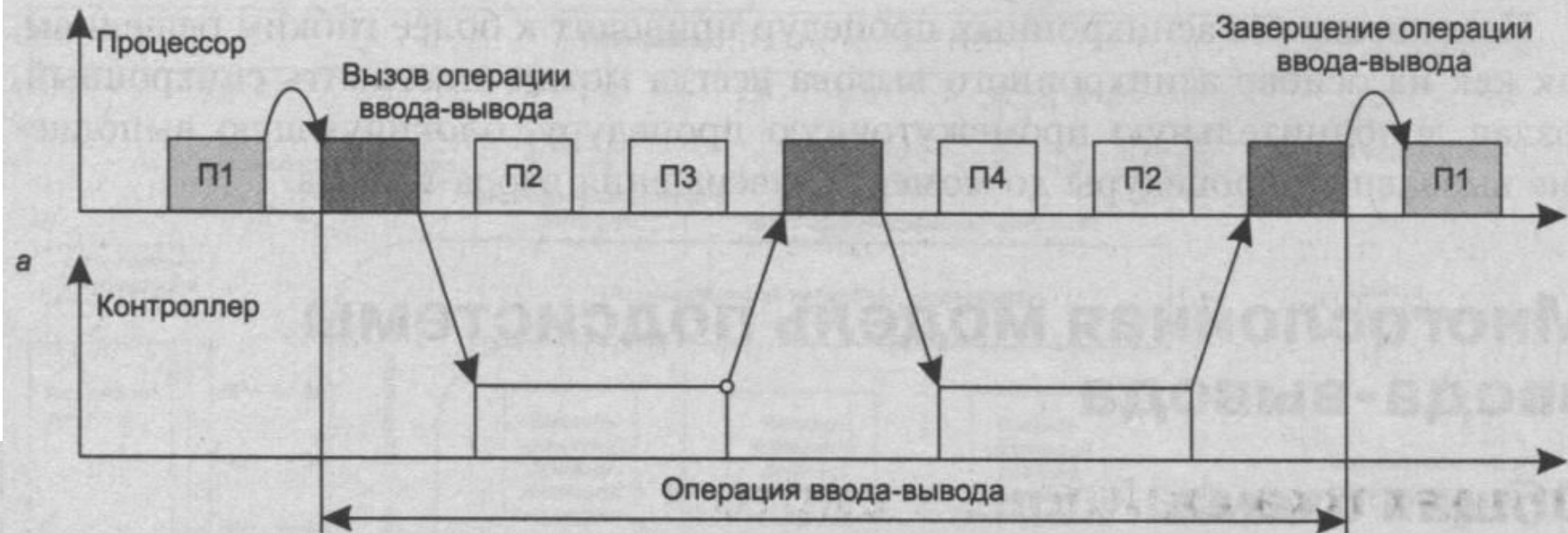
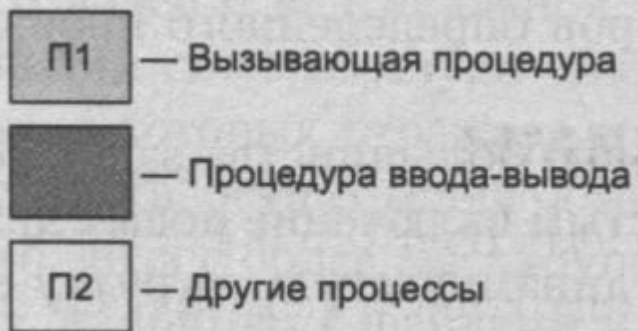
Обычно в операционной системе имеется специальный слой программного обеспечения, отвечающий за решение данной задачи, например слой **VFS (Virtual File System)** в версиях Unix на основе кода System V Release 4

Синхронный и асинхронный режимы

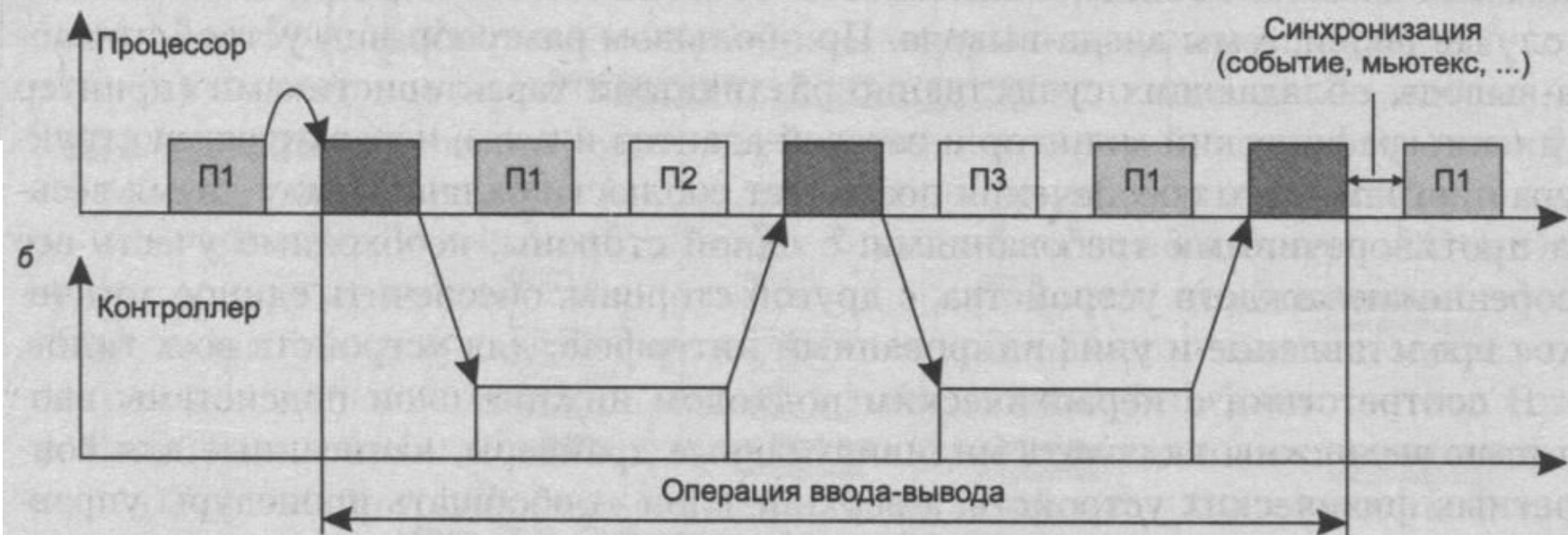
По отношению к программному модулю, запросившему операцию ввода-вывода, она может выполняться в **синхронном** или **асинхронном** режиме

Смысл этих режимов тот же, что и для рассмотренных ранее системных вызовов:

- В **синхронном режиме** программный модуль приостанавливает свою работу до тех пор, пока операция ввода-вывода не будет завершена (а)
- В **асинхронном режиме** программный модуль продолжает выполняться в мультипрограммной среде одновременно с операцией ввода-вывода (б)



Синхронное выполнение операции ввода-вывода



Асинхронное выполнение операции ввода-вывода

Синхронный и асинхронный режимы

Системные вызовы ввода-вывода чаще оформляются как **синхронные процедуры** в связи с тем, что такие операции длятся долго, и пользовательскому процессу или потоку все равно придется ждать получения результатов операции для того, чтобы продолжить свою работу

Внутренние же вызовы операций ввода-вывода из модулей ядра обычно выполняются в виде асинхронных процедур, так как кодам ядра нужна свобода в выборе дальнейшего поведения после запроса операции ввода-вывода

Синхронный и асинхронный режимы

Иногда и прикладному процессу может потребоваться выполнить асинхронную операцию ввода-вывода, например, если этот процесс является прикладным только по форме, а по сути он представляет собой часть микроядерной ОС. В таком случае ему необходима полная свобода действий и после того, как он вызвал операцию ввода-вывода

Использование асинхронных процедур приводит к более гибким решениям, так как на основе асинхронного вызова всегда можно построить синхронный, создав дополнительную промежуточную процедуру, блокирующую выполнение вызвавшей процедуры до момента завершения ввода-вывода

Многослойная модель подсистемы ввода-вывода

Многослойное построение программного обеспечения, характерное для операционных систем вообще, оказывается особенно естественным и полезным в случае подсистемы ввода-вывода

При большом разнообразии устройств ввода-вывода, обладающих существенно различными характеристиками (принтер и диски, графический монитор и сетевой адаптер и т. п.), иерархическая структура программного обеспечения позволяет соблюсти баланс между двумя весьма противоречивыми требованиями:

- С одной стороны, необходимо учесть все особенности каждого устройства
- С другой стороны, обеспечить единое логическое представление и унифицированный интерфейс для устройств всех типов

Многослойная модель подсистемы ввода-вывода

В соответствии с иерархическим подходом нижние слои подсистемы ввода-вывода должны включать индивидуальные драйверы, написанные для конкретных физических устройств

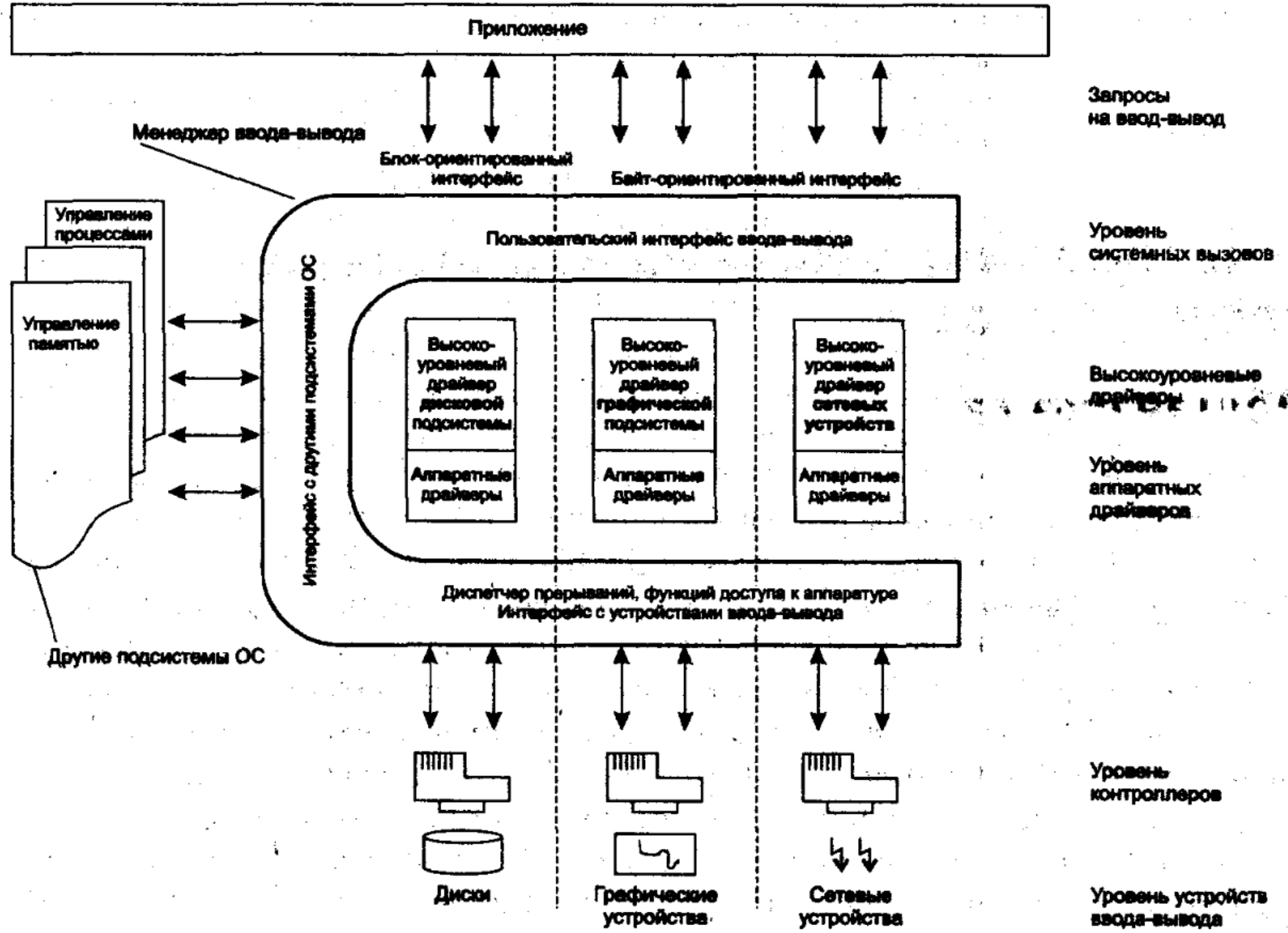
Верхние слои – обобщать процедуры управления этими устройствами, предоставляя общий интерфейс если не для всех устройств, то, по крайней мере, для групп устройств, обладающих некоторыми общими характеристиками, например, для принтеров определенного производителя или для всех матричных принтеров и т. п.

Многослойная модель подсистемы ввода-вывода

Многослойность структуры способствует решению большинства задан подсистемы ввода-вывода, делая, например более простым включение новых драйверов, поддержку нескольких файловых систем, динамическую загрузку-выгрузку драйверов и других

Из следующей схемы видно, что программное обеспечение ввода-вывода делится не только на горизонтальные слои, но и на вертикальные

Это объясняется тем, что для такого разнообразного мира, как внешние устройства, трудно обеспечить единообразие в разбиении функций управления на слои



Многослойная модель подсистемы ввода-вывода

Поэтому общий принцип многослойности остается справедливым, однако для устройств определенного типа он реализуется по-разному с разным количеством слоев и их функциями

В представленной структуре в качестве примера приведены три вертикальные подсистемы, управляющие дисками, графическими устройствами (такими как мониторы, принтеры и плоттеры) и сетевыми адаптерами

Естественно, к этому перечню можно добавить и другие, например, подсистему управления символьными терминалами или какими-либо специализированными устройствами, такими как аналого-цифровые и цифро-аналоговые преобразователи

Многослойная модель подсистемы ввода-вывода

В каждой вертикальной подсистеме существует несколько слоев модулей

Нижний слой образуют так называемые аппаратные драйверы устройств, название которых отражает тот факт, что они управляют аппаратурой внешних устройств, **осуществляя обмен байтами и блоками байтов**, и не имеют, как правило, дела с более высокоуровневыми вопросами логической организации данных, например с файлами или сложными графическими объектами

Функции вышележащих слоев в значительной степени зависят от типа вертикальной подсистемы

Менеджер ввода-вывода

В подсистеме ввода-вывода наряду с модулями, отражающими специфику внешних устройств и образующими вертикальные подсистемы, существуют модули универсального назначения

Эти модули организуют согласованную работу всех остальных компонентов подсистемы ввода-вывода и взаимодействие с пользовательскими процессами и другими подсистемами ОС

Так же как функции управления устройствами, эти организующие функции распределены по всем уровням, образуя оболочку, называемую **менеджером ввода-вывода**

Менеджер ввода-вывода

Задачи менеджера ввода-вывода могут быть сведены к поддержке четырех интерфейсов:

- Пользовательский интерфейс ввода-вывода
- Интерфейс с устройствами ввода-вывода
- Интерфейс с другими подсистемами ОС
- Внутренний интерфейс между компонентами подсистемы ввода-вывода

Менеджер ввода-вывода

Верхний слой менеджера составляют модули ОС, которые принимают от пользовательских процессов запросы на ввод-вывод, поступающие в виде **системных вызовов**, и переадресуют их отвечающим за определенный класс устройств модулям и драйверам, а также возвращают процессам результаты операций ввода-вывода

Таким образом, этот слой поддерживает **пользовательский интерфейс ввода-вывода**, создавая для прикладных программистов максимум удобств по манипулированию внешними устройствами и расположенными на них данными

Менеджер ввода-вывода

Нижний слой менеджера реализует непосредственное взаимодействие с контроллерами внешних устройств, экранируя драйверы от особенностей аппаратной платформы компьютера – шины ввода-вывода, системы прерываний и т. п.

Этот слой принимает от драйверов запросы на обмен данными с регистрами контроллеров в некоторой обобщенной форме с использованием независимых от шины ввода-вывода адресации и формата, а затем преобразует эти запросы в зависящий от аппаратной платформы формат

Диспетчер прерываний, рассмотренный ранее, может входить в состав менеджера ввода-вывода или же представлять собой отдельный модуль ядра. В последнем случае менеджер ввода-вывода выполняет для диспетчера прерываний первичную обработку запросов прерываний, передавая диспетчеру обобщенные сведения об источнике запроса

Менеджер ввода-вывода

Еще одной функцией менеджера является организация взаимодействия модулей подсистемы ввода-вывода с модулями других подсистем ОС, таких как подсистемы управления процессами, виртуальной памяти и другие

Важной функцией менеджера ввода-вывода является также создание некоторой удобной среды для взаимодействия компонентов подсистемы ввода-вывода

Для этого в менеджер ввода-вывода включают поддержку некоторого стандартного **внутреннего интерфейса** взаимодействия модулей ввода-вывода между собой, а также нагружают менеджер выполнением наиболее часто используемых при работе драйверов функций

Менеджер ввода-вывода

Эти функции оформляются как системные процедуры, которые драйвер может вызывать для выполнения некоторых типовых действий

Примерами могут служить операции обмена с регистрами контроллера, ведение буферов для промежуточного хранения данных ввода-вывода, синхронизация работы нескольких драйверов, копирование данных из пользовательского пространства в пространство системы.

Такой подход существенно облегчает разработку и включение новых драйверов и файловых систем в состав ОС

Менеджер ввода-вывода

Роль менеджера ввода-вывода выполняет среда STREAMS, существующая во многих версиях операционной системы Unix

Другим примером является менеджер ввода-вывода ОС семейства Windows NT. Он организует взаимодействие между модулями с помощью пакетов запросов ввода-вывода (I/O Request Packet, IRP). Получив запрос от процедуры системного вызова, менеджер формирует IRP и передает его нужному драйверу. Драйвер после выполнения запрошенной операции возвращает ответ в виде еще одного пакета IRP менеджеру, а тот, в свою очередь, может при необходимости передать этот пакет IRP другому драйверу

Менеджер позволяет драйверам задавать взаимосвязи (bindings) между собой, и на основании информации о взаимосвязях и происходит передача пакетов IRP. Кроме того, менеджер ОС семейства Windows NT поддерживает динамическую загрузку-выгрузку драйверов без останова системы

Менеджер ввода-вывода

Наличие стандартного внутреннего межмодульного интерфейса повышает устойчивость и улучшает расширяемость подсистемы ввода-вывода, хотя может несколько замедлить ее работу, так как любое разделение на слои и части приводит к дополнительным операциям взаимодействия по сравнению с монолитной организацией с прямыми передачами управления

Многоуровневые драйверы

Первоначально термин «драйвер» применялся в более узком смысле, чем в настоящее время:

Драйвер – это программный модуль, который:

- Входит в состав ядра операционной системы, работая в привилегированном режиме
- Непосредственно управляет внешним устройством, взаимодействуя с его контроллером с помощью команд ввода-вывода компьютера
- Обрабатывает прерывания от контроллера устройства
- Предоставляет прикладному программисту удобный логический интерфейс работы с устройством, экранируя от него низкоуровневые детали управления устройством и организации его данных
- Взаимодействует с другими модулями ядра ОС с помощью строго оговоренного интерфейса, описывающего формат передаваемых данных, структуру буферов, способы включения драйвера в состав ОС, способы вызова драйвера, набор общих процедур подсистемы ввода-вывода, которыми драйвер может пользоваться, и т. п.

Многоуровневые драйверы

Согласно этому определению драйвер вместе с контроллером устройства и прикладной программой воплощали идею многослойного подхода к организации программного обеспечения

Контроллер представлял нижний слой управления устройством, выполняющий операции в терминах блоков и агрегатов устройства (например, передвижение головки дисководов, побитная передача байта по двухпроводному кабелю)

Драйвер выполнял более сложные операции, преобразуя, например, данные, адресуемые в терминах номеров цилиндров, головок и секторов диска, в линейную последовательность блоков или устанавливая логическое соединение между двумя модемами через телефонную сеть

Многоуровневые драйверы

В результате прикладная программа уже работала с данными, преобразованными в достаточно понятную для человека форму – файлами, таблицами баз данных, текстовыми окнами на мониторе и т. п., не вдаваясь в детали представления этих данных в устройствах, ввода-вывода

Кроме того, помещение драйвера в привилегированный режим и запрет для пользовательских процессов выполнять операции ввода-вывода защищали критически важные для работы самой ОС устройства ввода-вывода от ошибок прикладных программ, а также позволяли ОС надежно контролировать процесс разделения устройств и их данных между пользователями и процессами

Многоуровневые драйверы

В описанной схеме драйверы не делились на слои. При этом они решали задачи разного уровня сложности, как самые примитивные, например, просто последовательно передавая контроллеру байты для дальнейшего использования, так и достаточно сложные, связанные с отработкой протокола взаимодействия между модемами или вычерчиванием на экране математических кривых

Постепенно, по мере развития операционных систем и усложнения структуры подсистемы ввода-вывода, наряду с традиционными драйверами в операционных системах появились так называемые **высокоуровневые драйверы**, которые располагаются в общей модели подсистемы ввода-вывода над традиционными драйверами

Многоуровневые драйверы

Появление высокоуровневых драйверов можно считать дальнейшим развитием идеи многослойной организации подсистемы ввода-вывода

Вместо того чтобы концентрировать все функции по управлению устройством в одном программном модуле, во многих случаях гораздо эффективней распределить их между несколькими модулями в соседних слоях иерархии

Традиционные драйверы, которые стали называть **аппаратными драйверами, низкоуровневыми драйверами** или **драйверами устройств** подчеркивая их непосредственную связь с управляемым устройством, освобождаются от высокоуровневых функций.

Выполняемые ими низкоуровневые операции составляют фундамент, на котором можно построить тот или иной набор операций в драйверах более высоких уровней

Многоуровневые драйверы

При таком подходе повышается гибкость и расширяемость функций по управлению устройством – вместо жесткого набора функций, сосредоточенных в единственном драйвере, администратор ОС может выбрать требуемый набор функций, установив нужный высокоуровневый драйвер

Если различным приложениям необходимо взаимодействовать с различными логическими моделями одного и того же физического устройства, то для этого достаточно установить в системе несколько высокоуровневых драйверов, работающих поверх одного аппаратного драйвера

Многоуровневые драйверы

Количество уровней драйверов в подсистеме ввода-вывода, обычно не ограничивается каким-либо пределом, но на практике чаще всего используют от двух до пяти – слишком большое количество уровней может снизить скорость, операций ввода-вывода

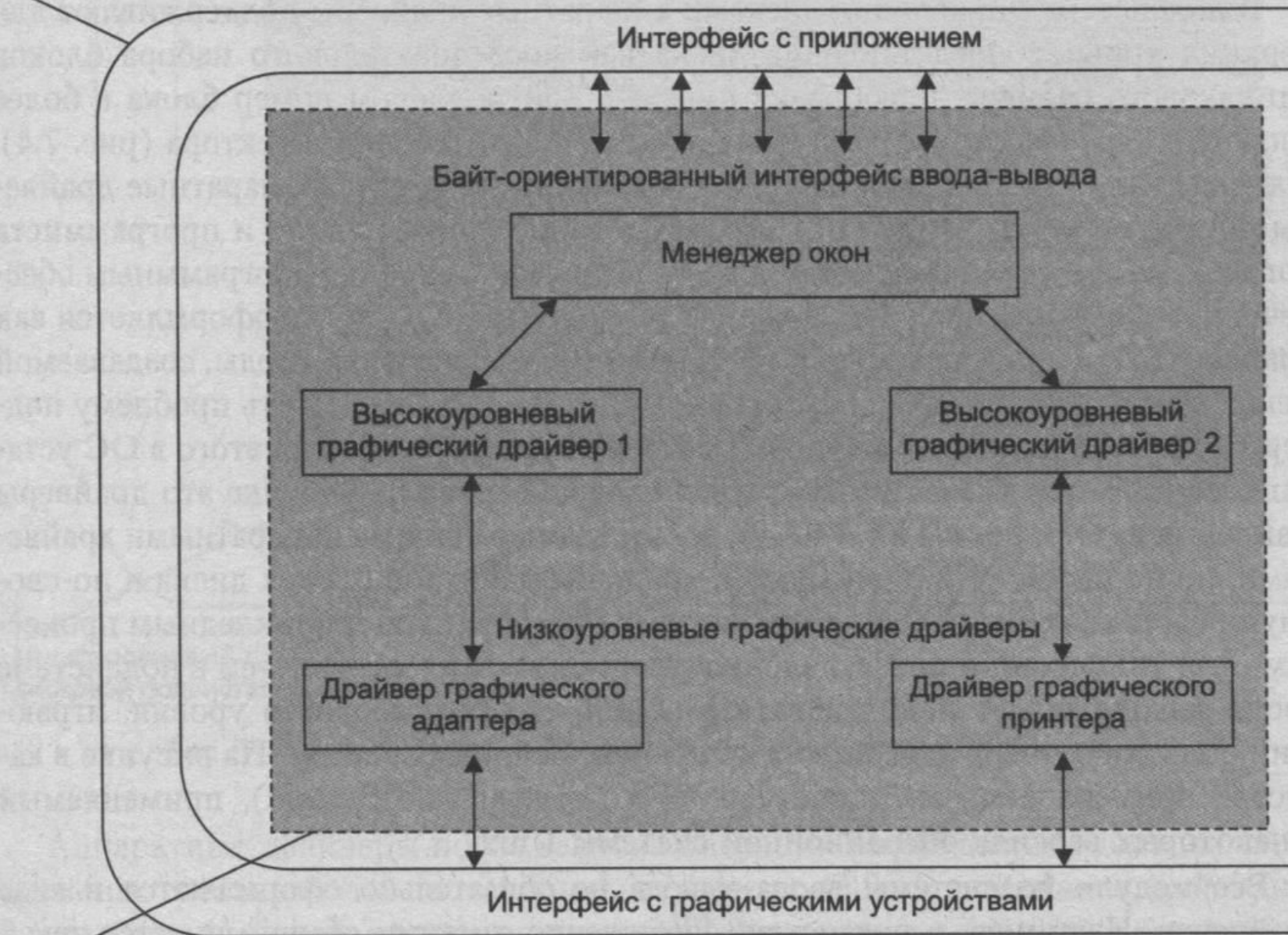
Несколько драйверов, управляющих одним устройством, но на разных уровнях, можно рассматривать как набор отдельных драйверов или как один **многоуровневый драйвер**

Многоуровневые драйверы

Высокоуровневые драйверы оформляются по тем же правилам и придерживаются тех же внутренних интерфейсов, что и аппаратные драйверы. Единственным отличием является то, что высокоуровневые драйверы, как правило, не вызываются по прерываниям, так как взаимодействуют с управляемым устройством через аппаратные драйверы

Менеджер ввода-вывода управляет драйверами однотипно независимо от того, к какому уровню он относится. При наличии большого количества драйверов разного уровня усложняются связи между ними, что, в свою очередь, усложняет их взаимодействие, и именно эта ситуация привела к стандартизации внутреннего интерфейса в подсистеме ввода-вывода и выделения специальной оболочки в виде менеджера ввода-вывода, выполняющего служебные функции по организации работы драйверов

Менеджер ввода-вывода



Многоуровневый драйвер
графической подсистемы
ввода-вывода

Многоуровневые драйверы

В подсистеме управления графическими устройствами, такими как графические мониторы и принтеры, существует несколько уровней драйверов

На нижнем уровне работают аппаратные драйверы, которые позволяют управлять конкретным графическим адаптером или принтером определенного типа, заставляя их выполнять некоторый набор примитивных графических операций: вывод точки, окружности, заполнение области цветом, вывод символов и т. п.

Многоуровневые драйверы

Высокоуровневые графические драйверы строят на базе этих операций более мощные операции, например масштабирование изображения, преобразование графического формата в соответствии с разрешающими возможностями устройства и т. п.

Самый верхний уровень подсистемы составляет менеджер окон, создающий для каждого приложения виртуальный образ экрана в виде набора окон, в которые приложение может выводить свои графические данные

Менеджер управляет окнами, отображая их в определенной области физического экрана или делая их невидимыми, а также предоставляет к ним совместный доступ с контролем прав доступа

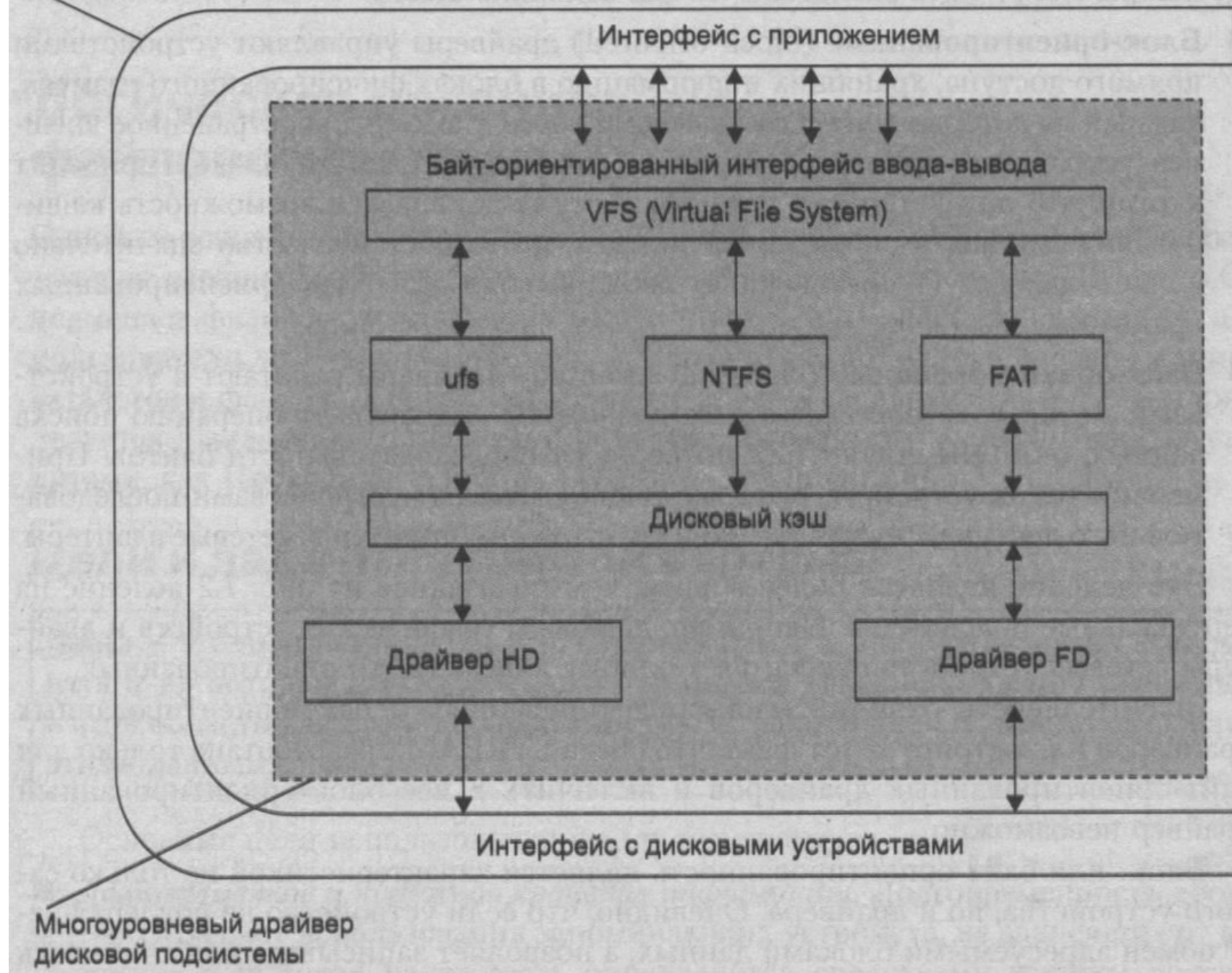
Менеджер окон уже не зависит от особенностей конкретного графического устройства, давая возможность высокоуровневым драйверам заниматься преобразованием форматов выводимых данных

Многоуровневые драйверы

В подсистеме управления дисками аппаратные драйверы поддерживают для верхних уровней представление диска как последовательного набора блоков одинакового размера, преобразуя вместе с контроллером номер блока в более сложный адрес, состоящий из номеров цилиндра, головки и сектора

Однако такие понятия, как «файл» и «файловая система», аппаратные драйверы дисков не поддерживают – эти удобные для пользователя и программиста логические абстракции создаются на более высоком уровне программным обеспечением файловых систем, которое в современных ОС также оформляется как драйвер, только высокоуровневый

Менеджер ввода-вывода



Многоуровневые драйверы

Наличие универсальной среды, создаваемой менеджером ввода-вывода, позволяет достаточно просто решить проблему поддержки в ОС нескольких файловых систем одновременно

Для этого в ОС устанавливается несколько высокоуровневых драйверов (на рисунке это драйверы файловых систем ufs, NTFS и FAT), работающих с общими аппаратными драйверами, но по-своему организующими хранение данных в блоках диска и по-своему представляющими файловую систему пользователю и прикладным процессам

Для унификации представления различных файловых систем в подсистеме ввода-вывода может использоваться общий драйвер верхнего уровня, играющий роль диспетчера нескольких драйверов файловых систем.

На рисунке в качестве примера показан диспетчер VFS (Virtual File System), применяемый в некоторых версиях операционной системы Unix

Многоуровневые драйверы

Все модули подсистемы ввода-вывода **не обязательно оформляются в виде драйверов**

Например, в подсистеме управления дисками обычно имеется такой модуль, как дисковый кэш, который служит для кэширования блоков дисковых файлов в оперативной памяти. Достаточно специфические функции кэша делают нецелесообразным оформление его в виде драйвера, взаимодействующего с другими модулями ОС только с помощью менеджера ввода-вывода

Другим примером модуля, который чаще всего не оформляется в виде драйвера, является диспетчер окон графического интерфейса. Иногда этот модуль вообще выносится из ядра ОС и реализуется в виде пользовательского процесса

Многоуровневые драйверы

Аппаратные драйверы после запуска операции ввода-вывода должны своевременно реагировать на завершение контроллером заданного действия, и для решения этой задачи они взаимодействуют с системой прерываний

Драйверы более высоких уровней вызываются уже не по прерываниям, а по инициативе аппаратных драйверов или драйверов вышележащего уровня

Не все процедуры аппаратного драйвера нужно вызывать по прерываниям, поэтому драйвер обычно имеет определенную структуру, в которой выделяется **процедура обслуживания прерываний** (Interrupt Service Routine, ISR), – она и вызывается диспетчером прерываний при поступлении запроса от соответствующего устройства

Диспетчер прерываний можно считать частью подсистемы ввода-вывода, а можно считать и независимым модулем ядра ОС, так как он служит не только для вызова процедур обслуживания прерываний драйверов, но и для диспетчеризации прерываний других типов

Виды драйверов

Драйверы могут быть разделены на два больших класса: **блок-ориентированные** и **байт-ориентированные**

Блок-ориентированные (block-oriented) драйверы управляют устройствами прямого доступа, хранящих информацию в блоках фиксированного размера, каждый из которых ***имеет собственный адрес***

Самое распространенное внешнее устройство прямого доступа – диск

Адресуемость блоков приводит к тому, что для устройств прямого доступа появляется возможность кэширования данных в оперативной памяти, и это обстоятельство значительно влияет на общую организацию ввода-вывода для блок-ориентированных драйверов

Виды драйверов

Байт-ориентированные (character-oriented) драйверы работают с устройствами, которые ***не адресуемы*** и не позволяют производить операцию поиска данных, они генерируют или потребляют последовательности байтов

Примерами таких устройств, которые также называют устройствами последовательного доступа, служат терминалы, строчные принтеры, сетевые адаптеры

Это деление является более общим, чем показанное на схеме деление на вертикальные подсистемы

Например, драйверы графических устройств и драйверы сетевых устройств относятся к одному классу байт-ориентированных

Виды драйверов

Значительность отличий блок-ориентированных и байт-ориентированных драйверов иллюстрирует тот факт, что среда STREAMS разработана только для байт-ориентированных драйверов и включить в нее блок-ориентированный драйвер невозможно

Блок- или байт-ориентированность является характеристикой не только самого устройства, но и драйвера. Очевидно, что если устройство не поддерживает обмен адресуемыми блоками данных, а позволяет записывать или считывать последовательность байтов, то и устройство, и его драйвер можно назвать байт-ориентированными

Виды драйверов

Для байт-ориентированного устройства невозможно разработать блок-ориентированный драйвер. Однако блок-ориентированным устройством можно управлять и с помощью байт-ориентированного драйвера

Так, диск можно рассматривать не только как набор блоков, но и набор байтов, первый из которых начинает первый блок диска, а последний завершает последний блок

Физический обмен с контроллером устройства по-прежнему осуществляется блоками, но байт-ориентированный драйвер устройства будет преобразовывать блоки в последовательности байтов. Для устройств прямого доступа часто разрабатывают пару драйверов, чтобы к устройству можно было обращаться к по байт-ориентированному, и по блок-ориентированному интерфейсам, в зависимости от потребностей

Виды драйверов

Деление всех драйверов на блок-ориентированные и байт-ориентированные оказывается полезным для структурирования подсистемы управления вводом-выводом: тем не менее необходимо учитывать, что эта схема является упрощенной – имеются внешние устройства, драйверы которых не относятся ни к одному из указанных классов

Таковым, например, является таймер, который, с одной стороны, не содержит адресуемой информации, а с другой стороны, не порождает потока байтов. Это устройство только выдает сигнал прерывания в некоторые моменты времени

Логическая организация файловой системы

Одной из основных задач операционной системы является предоставление пользователю удобных средств работы с данными, хранящимися на дисках

Для этого ОС подменяет физическую структуру хранящихся данных некоторой удобной для пользователя логической моделью – системой иерархически организованных каталогов и файлов

Для пользователя это абстрактное представление материализуется в виде набора ярлыков или списка, выводимого на экран такими утилитами, как Проводник Windows (Windows Explorer)

Цели и задачи файловой системы

Файл – это именованная область внешней памяти, в которую можно записывать и из которой можно считывать данные, а также собственно хранимые в этой области данные и набор атрибутов, позволяющих ОС манипулировать этими данными

Основные цели использования файла перечислены ниже:

- Долговременное и надежное хранение информации
- Совместное использование информации

Цели и задачи файловой системы

Долговременное и надежное хранение информации.

Долговременность достигается за счет использования запоминающих устройств, не зависящих от питания*, а высокая надежность определяется средствами защиты доступа к файлам и общей организацией программного кода ОС, при которой сбои аппаратуры чаще всего не разрушают информацию, хранящуюся в файлах

*Файлы хранятся в памяти, не зависящей от энергопитания, однако имеется исключение – так называемый электронный диск, представляющий собой созданную в оперативной памяти структуру, имитирующую файловую систему

Цели и задачи файловой системы

Совместное использование информации

Файлы обеспечивают естественный и простой способ разделения информации между приложениями и пользователями за счет наличия понятного человеку символьного имени и постоянства хранимой информации и расположения файла

Пользователь должен иметь удобные средства работы с файлами, включая, каталоги-справочники, объединяющие файлы в группы, средства поиска файлов по признакам, набор команд для создания, модификации и удаления файлов

Файл может быть создан одним пользователем, а применяться совсем другим, при этом создатель файла или администратор могут определить права доступа к нему других пользователей. Эти цели реализуются в ОС **файловой системой**

Цели и задачи файловой системы

Файловая система (ФС) – это часть операционной системы, включающая:

- Совокупность всех файлов на диске
- Наборы структур данных, используемых для управления файлами, такие, например, как каталоги файлов, дескрипторы файлов, таблицы распределения свободного и занятого пространства на диске
- Комплекс системных программных средств, реализующих различные операции с файлами, такие как создание, уничтожение, чтение, запись, именованное и поиск файлов

Цели и задачи файловой системы

Файловая система позволяет программам обходиться набором достаточно простых операций для выполнения действий над некоторым абстрактным объектом, представляющим файл

При этом программистам не нужно иметь дело с деталями действительного расположения данных на диске, буферизацией данных и другими низкоуровневыми проблемами передачи данных с долговременного запоминающего устройства. Все эти функции файловая система берет на себя

Файловая система распределяет дисковую память, поддерживает именование файлов, отображает имена файлов на соответствующие адреса во внешней памяти, обеспечивает доступ к данным, поддерживает разделение, защиту и восстановление файлов

Цели и задачи файловой системы

Таким образом, файловая система играет роль промежуточного слоя, экранирующего все сложности физической организации долговременного хранилища данных и создающего для программ более простую логическую модель этого хранилища, а также предоставляя им набор удобных в использовании команд для манипулирования файлами

Задачи, решаемые ФС, зависят от способа организации вычислительного процесса в целом

Цели и задачи файловой системы

Самый простой тип – это ФС в однопользовательских и однопользовательских ОС, к числу которых относится, например, MS-DOS. Основные функции в **однопользовательских однопользовательских ФС** нацелены на решение следующих задач:

- Именованное хранение файлов
- Программный интерфейс для приложений
- Отображение логической модели файловой системы на физическую организацию хранилища данных
- Устойчивость файловой системы к сбоям питания, ошибкам аппаратных и программных средств

Цели и задачи файловой системы

Задачи ФС усложняются в операционных ***однопользовательских мультипрограммных ОС***, которые, хотя и предназначены для работы одного пользователя, но дают ему возможность запускать одновременно несколько процессов

Одной из первых ОС этого типа стала OS/2

К перечисленным задачам добавляется новая задача совместного доступа к файлу из нескольких процессов

Файл в этом случае является разделяемым ресурсом, а, значит, файловая система должна решать весь комплекс проблем, связанных с такими ресурсами

Цели и задачи файловой системы

В частности, в однопользовательских мультипрограммных ФС должны быть предусмотрены средства блокировки файла и его частей, предотвращения гонок, исключение тупиков, согласование копий и т. п.

В **многопользовательских системах ФС** появляется еще одна задача – защита файлов одного пользователя от несанкционированного доступа другого пользователя

Еще более сложными становятся функции ФС, которая работает в составе сетевой ОС

Типы файлов

Файловые системы поддерживают несколько функционально различных типов файлов, в число которых, как правило, входят:

- Обычные файлы
- Файлы-каталоги
- Специальные файлы
- Именованные конвейеры
- Файлы, отображаемые на память

Типы файлов

Обычные файлы, или просто **файлы**, содержат информацию произвольного характера, которую заносит в них пользователь или которая образуется в результате работы системных и пользовательских программ

Большинство современных операционных систем (например, Unix, Windows) никак не ограничивают и не контролируют содержимое и структуру обычного файла

Содержимое обычного файла определяется приложением, которое с ним работает

Например, текстовый редактор создает текстовые файлы, состоящие из строк символов, представленных в каком-либо коде. Это могут быть документы, исходные тексты программ и т. п. Текстовые файлы можно прочитать на экране и распечатать на принтере

Двоичные файлы не используют коды символов, они часто имеют сложную внутреннюю структуру, например исполняемый код программы или архивный файл

Все операционные системы должны уметь распознавать хотя бы один тип файлов – их **собственные исполняемые файлы**

Типы файлов

Каталоги – это особый тип файлов, которые содержат системную справочную информацию о наборе файлов, сгруппированных пользователями по какому-либо неформальному признаку (например, в одну группу объединяются файлы, содержащие документы одного договора, или файлы, составляющие один программный пакет)

Во многих операционных системах в каталог могут входить файлы любых типов, в том числе другие каталоги, за счет чего образуется древовидная структура, удобная для поиска

Каталоги устанавливают соответствие между именами файлов и их характеристиками, используемыми файловой системой для управления файлами. В число таких характеристик входит, в частности, информация (или указатель на другую структуру, содержащую эти данные) о типе файла и расположении его на диске, правах доступа к файлу и датах его создания и модификации. Во всех остальных отношениях каталоги рассматриваются файловой системой как обычные файлы

Типы файлов

Специальные файлы – это фиктивные файлы, ассоциированные с устройствами ввода-вывода, которые используются для унификации механизма доступа к файлам и внешним устройствам.

Специальные файлы позволяют пользователю выполнять операции ввода-вывода посредством обычных команд записи в файл или чтения из файла

Эти команды обрабатываются сначала программами файловой системы, а затем на некотором этапе выполнения запроса преобразуются операционной системой в команды управления соответствующим устройством

Современные файловые системы поддерживают и другие типы файлов, такие как символьные связи, именованные конвейеры, отображаемые на память файлы

Типы файлов

Примеры специальных файлов:

Windows	Linux
COM1, COM2, COM3, COM4, COM5, COM6, COM7, COM8, COM9	/dev/ttyS0, /dev/ttyS1, /dev/ttyS2...
LPT1, LPT2, LPT3, LPT4, LPT5, LPT6, LPT7, LPT8, LPT9	/dev/lp0, /dev/lp1, /dev/lp2
CON	/dev/tty1, /dev/tty2...
PRN	
NUL	/dev/null
AUX	

Иерархическая структура файловой системы

Пользователи обращаются к файлам по символьным именам. Однако способности человеческой памяти ограничивают количество имен объектов, к которым пользователь может обращаться по имени

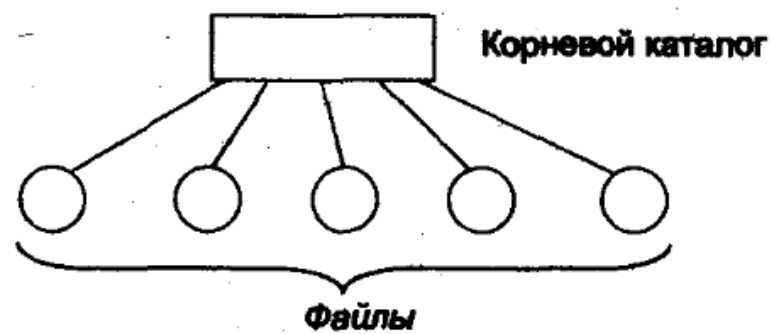
Иерархическая организация пространства имен позволяет значительно расширить эти границы. Именно поэтому большинство файловых систем имеет иерархическую структуру, в которой уровни создаются за счет того, что каталог более низкого уровня может входить в каталог более высокого уровня

При этом частным случаем иерархической структуры является одноуровневая организация, когда все файлы входят в один каталог (а)

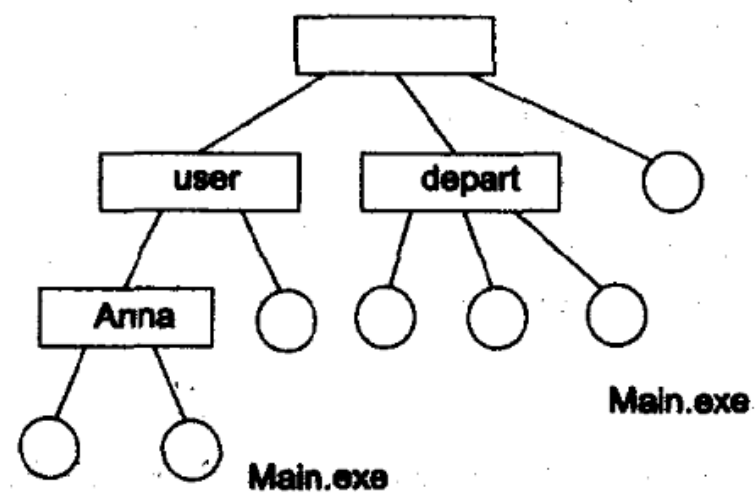
Граф, описывающий иерархию каталогов, может быть деревом (б) или сетью (в)

Каталоги образуют **дерево**, если файлу разрешено входить только в один каталог, и **сеть**, если файл может входить сразу в несколько каталогов

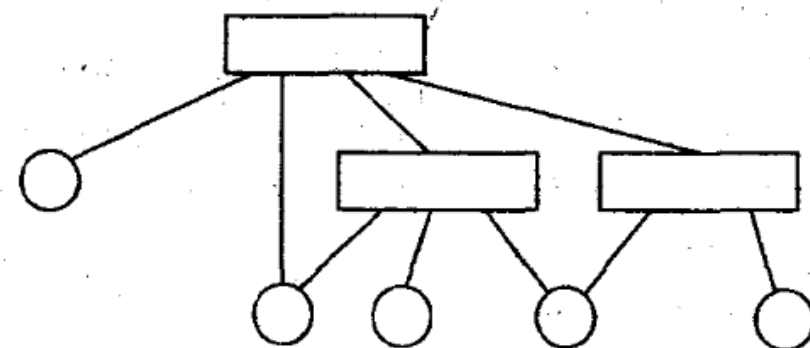
а



б



в



Иерархическая структура файловой системы

Например, в MS-DOS и Windows каталоги образуют древовидную структуру, а в Unix – сетевую

В древовидной структуре каждый файл является **листом**. Каталог самого верхнего уровня называется **корневым каталогом**, или **корнем** (root)

При такой организации пользователь освобожден от необходимости помнить имена всех файлов, ему достаточно примерно представлять, к какой группе может быть отнесен тот или иной файл, чтобы путем последовательного просмотра каталогов найти его

Иерархическая структура удобна для многопользовательской работы: каждый пользователь со своими файлами локализуется в своем каталоге или поддереве каталогов, и вместе с тем все файлы в системе логически связаны

Имена файлов

Все типы файлов имеют символьные имена.

В иерархически организованных файловых системах обычно используются следующие типы имен файлов:

- Простое (короткое) символьное имя
- Полное (составное) символьное имя
- Относительное символьное имя
- Уникальное имя (числовой идентификатор)

Имена файлов

Простое, или короткое, символьное имя идентифицирует файл в пределах одного каталога

Простые имена присваивают файлам пользователи и программисты, при этом они должны учитывать ограничения ОС как на номенклатуру символов, так и на длину имени

До сравнительно недавнего времени эти границы были весьма узкими. Так, в известной файловой системе FAT MS-DOS длина имен ограничивалась схемой 8.3 (8 символов – собственно имя, 3 символа – расширение имени), а в файловой системе s5, поддерживаемой многими версиями ОС Unix, простое символьное имя не могло содержать более 14 символов

Имена файлов

Однако пользователю гораздо удобнее работать с длинными именами, поскольку они позволяют давать файлам легко запоминающиеся названия, ясно говорящие о том, что содержится в том или ином файле

Поэтому современные файловые системы, а также усовершенствованные варианты уже существовавших файловых систем, как правило, поддерживают длинные простые символьные имена файлов. Например, в файловых системах NTFS и FAT32, входящих в состав ОС семейства Windows, имя файла может содержать до 255 символов

Имена файлов

Примеры простых имен файлов и каталогов:

- quest_ul.doc
- task-entran.exe
- приложение к CD 254L на русском языке.doc
- installable filesystem manager.doc

В иерархических файловых системах разным файлам разрешено иметь одинаковые простые символьные имена при условии, что они принадлежат разным каталогам

То есть здесь работает схема ***много файлов – одно простое имя***. Для однозначной идентификации файла в таких системах используется так называемое **полное имя**

Имена файлов

Полное имя представляет собой цепочку простых символьных имен всех каталогов, через которые проходит путь от корня до данного файла

Таким образом, полное имя является **составным именем**, в котором простые имена отделены друг от друга принятым в ОС разделителем

Часто в качестве разделителя используется прямой или обратный слэш, при этом принято не указывать имя корневого каталога. На рисунке два файла имеют простое имя Main.exe, однако их составные имена /depart/Main.exe и /user/Anna/Main.exe различаются

Имена файлов

В древовидной файловой системе между файлом и его полным именем имеется взаимно однозначное соответствие ***один файл – одно полное имя***

В файловых системах имеющих сетевую структуру, файл может входить в несколько каталогов, а значит, иметь несколько полных имен, здесь справедливо соответствие ***один файл – много полных имен***

В обоих случаях файл однозначно идентифицируется полным именем. Файл может быть идентифицирован также **относительным именем**

Имена файлов

Относительное имя файла определяется через понятие «текущий каталог»

Для каждого пользователя в каждый момент времени один из каталогов файловой системы является текущим, причем этот каталог выбирается самим пользователем по команде ОС

Файловая система фиксирует имя текущего каталога, чтобы затем использовать его как дополнение к относительным именам для образования полного имени файла. При применении относительных имен пользователь идентифицирует файл цепочкой имен каталогов, через которые проходят маршрут от текущего каталога до данного файла

Например, если текущим каталогом является каталог `/user`, то относительное имя файла `/user/Anna/Main.exe` выглядит следующим образом: `Anna/Main.exe`

Имена файлов

В некоторых операционных системах разрешено присваивать одному и тому же файлу несколько простых имен, которые можно интерпретировать как псевдонимы. В этом случае, так же как в системе с сетевой структурой, устанавливается соответствие ***один файл – много полных имен***, так как каждому простому имени файла соответствует, по крайней мере, одно полное имя.

Символьные имена удобны для пользователя, но не для операционной системы. Для своих внутренних целей ОС присваивает файлу **уникальное имя**, так что справедливо соотношение ***один файл – одно уникальное имя***

Уникальное имя существует наряду с одним или несколькими символьными именами, присваиваемыми файлу пользователями или приложениями. Уникальное имя представляет собой числовой идентификатор и предназначено только для использования операционной системой. Примером такого уникального имени файла является **номер индексного дескриптора (inode)** в системе Unix

Монтирование

В общем случае вычислительная система может иметь несколько дисковых устройств. Даже типичный персональный компьютер обычно имеет один накопитель на жестком диске, один накопитель на гибких дисках и накопитель для компакт-дисков

Мощные же компьютеры, как правило, оснащены большим количеством дисковых накопителей, на которые устанавливаются пакеты дисков

Более того, даже одно физическое устройство с помощью средств операционной системы может быть представлено в виде нескольких логических устройств, в частности, путем разбиения дискового пространства на разделы

Возникает вопрос, каким образом организовать хранение файлов в системе, имеющей несколько устройств внешней памяти?

Монтирование

Первое решение состоит в том, что на каждом из устройств размещается автономная файловая система, то есть файлы, находящиеся на этом устройстве описываются деревом каталогов, никак не связанным с деревьями каталогов на других устройствах. В таком случае для однозначной идентификации файла пользователь наряду с составным символьным именем файла должен указывать идентификатор логического устройства

Примером такого автономного существования файловых систем является операционная система MS-DOS, в которой полное имя файла включает буквенный идентификатор логического диска

Так, при обращении к файлу, расположенному на диске A, пользователь должен указать имя этого диска:

A:\privat\letter\uni\letl.doc1

Монтирование

Другим вариантом является такая организация хранения файлов, при которой пользователю предоставляется возможность объединять файловые системы, находящиеся на разных устройствах, в единую файловую систему, описываемую единым деревом каталогов

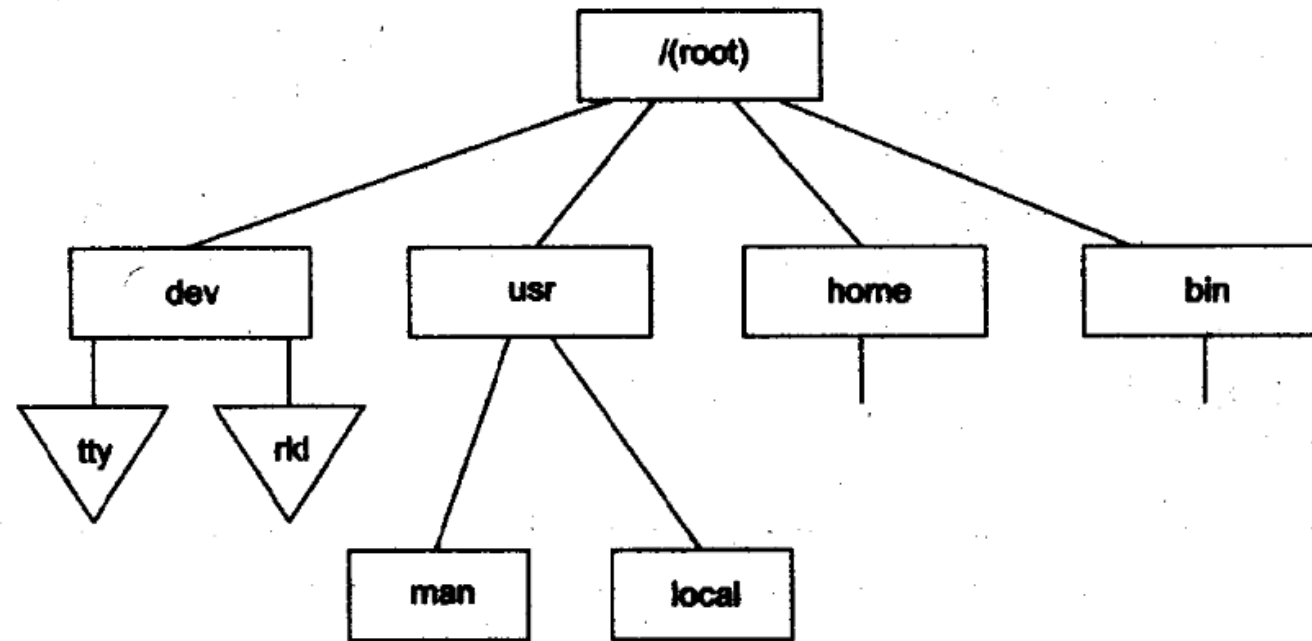
Такая операция называется **монтированием**

Рассмотрим, как осуществляется эта операция на примере ОС Unix

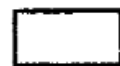
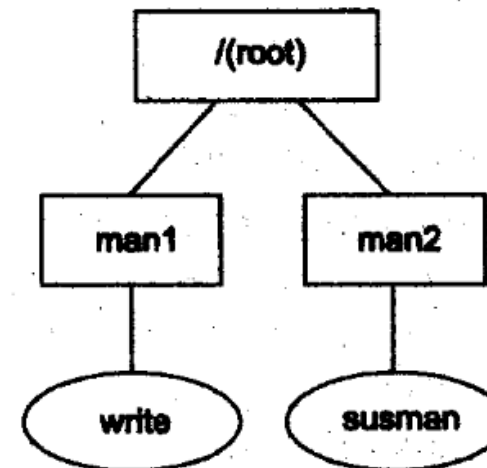
Среди всех имеющихся в системе логических дисковых устройств операционная система выделяет одно устройство, называемое системным

Пусть имеются две файловые системы, расположенные на разных логических дисках, причем один из дисков является системным

Файловая система 1



Файловая система 2



— Каталог



— Обычный файл



— Специальный файл-устройство

Монтирование

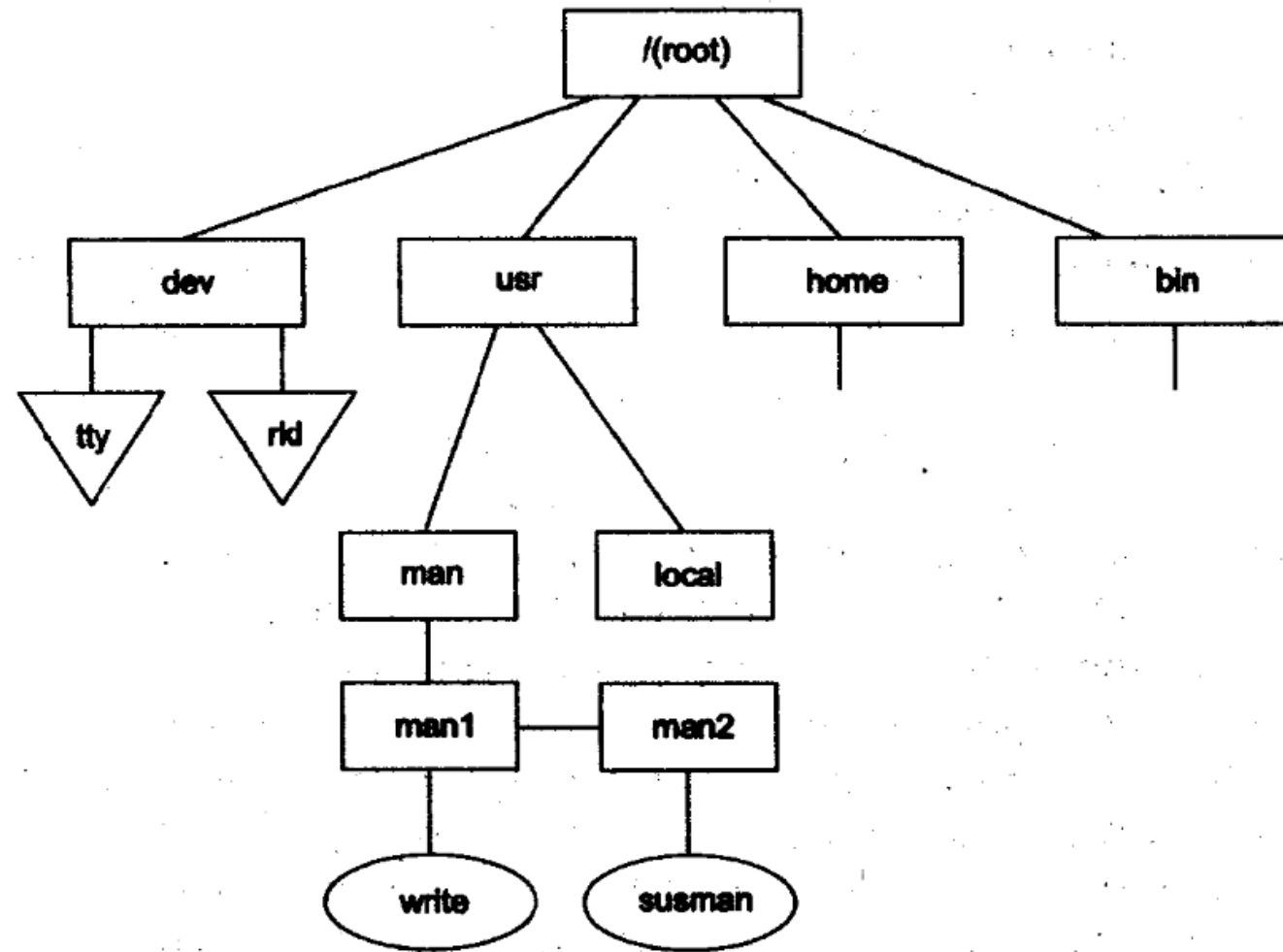
Файловая система, расположенная на системном диске, назначается **корневой**

Для связи иерархий файлов в корневой файловой системе выбирается некоторый существующий каталог, в данном примере – каталог `map`

После выполнения монтирования выбранный каталог `map` становится корневым каталогом второй файловой системы.

Через этот каталог монтируемая файловая система подсоединяется как поддерево к общему дереву

После монтирования общей файловой системы для пользователя нет логической разницы между корневой и смонтированной файловыми системами, в частности, именование файлов производится так же, как если бы система с самого начала была единой



Атрибуты файлов

Понятие «файл» включает не только хранимые им данные и имя, но и атрибуты. **Атрибуты** – это информация, описывающая свойства файла. Примеры возможных атрибутов файла:

- Тип файла (обычный файл, каталог, специальный файл и т. п.)
- Владелец файла
- Создатель файла
- Пароль для доступа к файлу
- Информация о разрешенных операциях доступа к файлу
- Времена создания, последнего доступа и последнего изменения
- Текущий размер файла
- Максимальный размер файла
- Признак «только для чтения»
- Признак «скрытый файл»
- Признак «системный файл»
- Признак «архивный файл»
- Признак «двоичный/символьный»
- Признак «временный» (удалить после завершения процесса)
- Признак блокировки
- Длина записи в файле
- Указатель на ключевое поле в записи
- Длина ключа

Атрибуты файлов

Набор атрибутов файла определяется спецификой файловой системы: в файловых системах разного типа для характеристики файлов могут использоваться разные наборы атрибутов

Например, в файловых системах, поддерживающих неструктурированные файлы, нет необходимости в трех последних атрибутах из приведенного списка, поскольку они связаны со структуризацией файла

В однопользовательской ОС в наборе атрибутов будут отсутствовать атрибуты, имеющие отношение к пользователям и защите, такие как владелец файла, создатель файла, пароль для доступа к файлу, информация о разрешенном доступе к файлу

Атрибуты файлов

Пользователь может получать доступ к атрибутам с помощью средств, предоставленных для этих целей файловой системой

Обычно разрешается читать значения любых атрибутов, а изменять – только некоторые

Например, пользователь может изменить права доступа к файлу (при условии, что он обладает необходимыми для этого полномочиями), но изменять дату создания или текущий размер файла ему не разрешается

Атрибуты файлов

Значения атрибутов файлов могут непосредственно содержаться в каталогах, как это было сделано в файловой системе MS-DOS. На рисунке представлена структура записи в каталоге, содержащая простое символьное имя и атрибуты файла

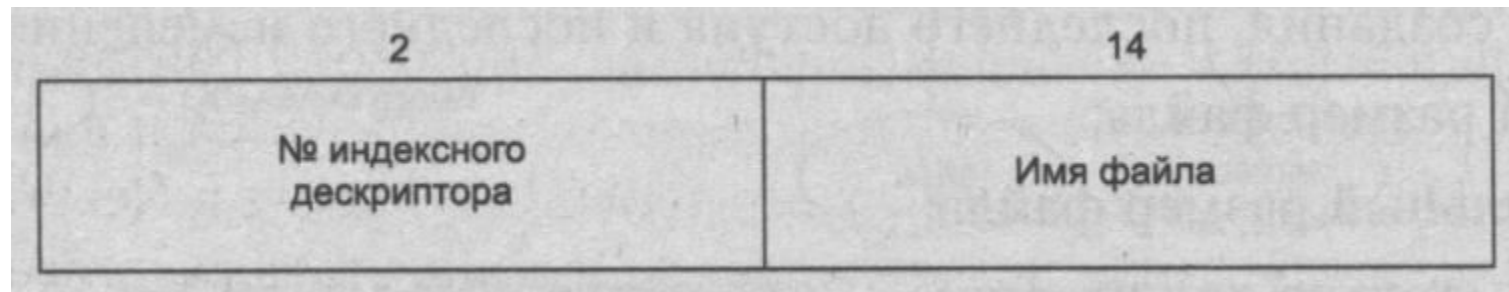
Здесь буквами обозначены признаки файла: R – только для чтения, A – архивный, H – скрытый, S – системный

8		3	1				4
Имя файла		Расширение	R	A	H	S	Резервные
Резервные	Время	Дата	№ первого кластера		Размер		

Атрибуты файлов

Другим вариантом является размещение атрибутов в специальных таблицах, когда в каталогах содержатся только ссылки на эти таблицы

Такой подход реализован, например, в файловой системе ufs ОС Unix. В этой файловой системе структура каталога очень простая. Запись о каждом файле содержит короткое символьное имя файла и указатель на **индексный дескриптор файла (inode)** – так называется в ufs таблица, в которой сосредоточены значения атрибутов файла



Атрибуты файлов

В том и другом вариантах каталоги обеспечивают связь между именами файлов и собственно файлами

Однако подход, когда имя файла отделено от его атрибутов, делает систему более гибкой

Например, файл может быть легко включен сразу в несколько каталогов. Записи об этом файле в разных каталогах могут содержать разные простые имена, но в поле ссылки будет указан один и тот же номер индексного дескриптора

Логическая организация файла

В общем случае данные, содержащиеся в файле, имеют некую **логическую структуру**. Эта структура является базой при разработке программы, предназначенной для обработки этих данных

Например, чтобы текст мог быть правильно выведен на экран, программа должна иметь возможность выделить отдельные слова, строки, абзацы и т. д. Признаками, отделяющими один структурный элемент от другого, могут служить определенные кодовые последовательности или просто известные программе значения смещений этих структурных элементов относительно начала файла

Поддержание структуры данных может быть либо целиком возложено на приложение, либо в той или иной степени эту работу может взять на себя файловая система

Логическая организация файла

В первом случае, когда все действия, связанные со структуризацией и интерпретацией содержимого файла, целиком относятся к ведению приложения, файл представляется ФС **неструктурированной** последовательностью данных

Приложение формулирует запросы к файловой системе на ввод-вывод, используя общие для всех приложений системные средства, например, указывая смещение от начала файла и количество байтов, которые необходимо считать или записать

Поступивший к приложению поток байтов интерпретируется в соответствии с заложенной в программе логикой

Подчеркнем, что интерпретация данных никак не связана с действительным способом их хранения в файловой системе

Логическая организация файла

Модель файла, в соответствии с которой содержимое файла представляется неструктурированной последовательностью (поток) байтов, стала популярной вместе с ОС Unix, а теперь она широко используется и в большинстве современных ОС, в том числе во всех версиях ОС Windows и NetWare

Неструктурированная модель файла позволяет легко организовать разделение файла между несколькими приложениями: разные приложения могут по-своему структурировать и интерпретировать данные, содержащиеся в файле

Логическая организация файла

Другая модель файла, которая применялась в ОС OS/360, DEC RSX и VMS, а в настоящее время используется достаточно редко, – это **структурированный** файл

В этом случае поддержание структуры файла поручается файловой системе. Файловая система видит файл как упорядоченную последовательность **логических записей**

Приложение может обращаться к ФС с запросами на ввод-вывод на уровне записей, например, «считать запись 25 из файла FILE.DOC». Файловая система должна обладать информацией о структуре файла, достаточной для того, чтобы выделить любую запись. ФС предоставляет приложению доступ к записи, а вся дальнейшая обработка данных, содержащихся в этой записи, выполняется приложением

Развитием этого подхода стали системы управления базами данных (СУБД), которые поддерживают не только сложную структуру данных, но и взаимосвязи между ними

Логическая организация файла

Логическая запись является наименьшим элементом данных, которым может оперировать программист при организации обмена с внешним устройством

Даже если физический обмен с устройством осуществляется большими единицами, операционная система должна обеспечивать программисту доступ к отдельной логической записи

Файловая система может использовать два способа доступа к логическим записям: читать или записывать логические записи последовательно (**последовательный доступ**) или позиционировать файл на запись с указанным номером (**прямой доступ**)

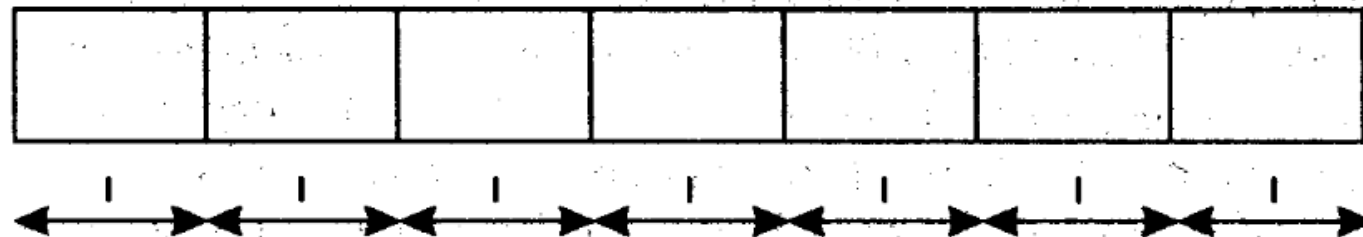
Логическая организация файла

Очевидно, что ОС не может поддерживать все возможные способы структурирования данных в файле, поэтому в тех ОС, в которых вообще поддерживается логическая структуризация файлов, поддержка реализована для небольшого числа широко распространенных схем логической организации файла

К числу таких способов структуризации относится представление данных в виде **записей фиксированной длины** (а). В этом случае доступ к n -й записи осуществляется либо путем последовательного чтения $(n - 1)$ предшествующих записей, либо прямо по адресу, вычисленному по ее порядковому номеру

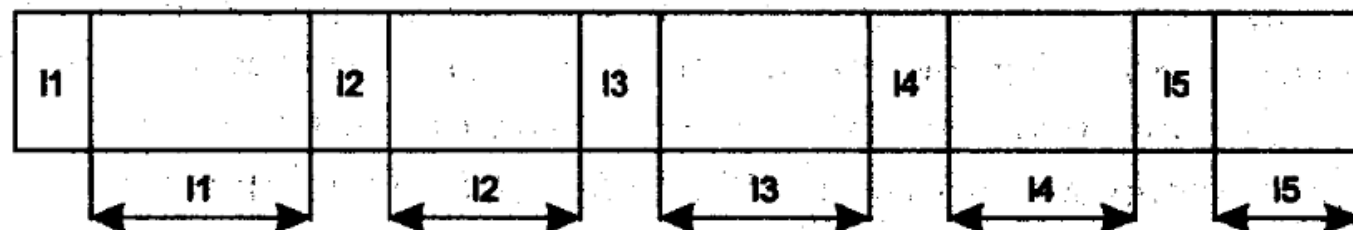
Например, если L – длина записи, то начальный адрес n -й записи равен $L * n$. Заметим, что при такой логической организации размер записи фиксирован в пределах файла, а записи в различных файлах, принадлежащих одной и той же файловой системе, могут иметь различный размер

а



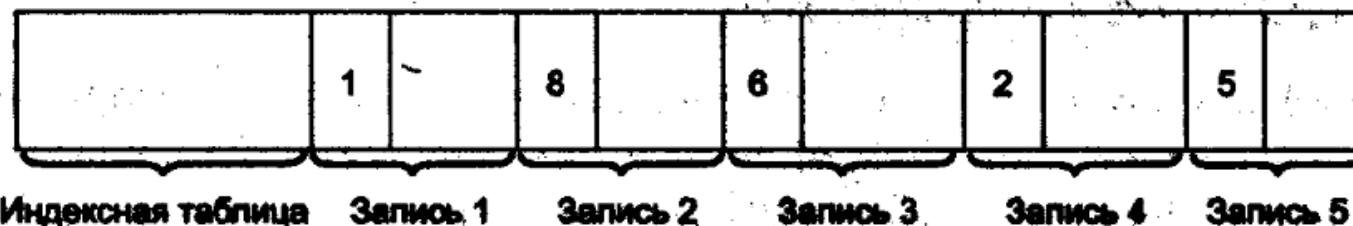
Последовательность логических записей фиксированной длины

б



Последовательность логических записей переменной длины

в



Индексная логическая организация

Индекс	1	2	3	4	5	6
Адрес	21	201	315	661	670	715

Индекс $\equiv$ ключ

Логическая организация файла

Другой способ структуризации состоит в представлении данных в виде **последовательности записей переменного размера**. Если расположить значения длин записей так, как это показано на рисунке (б), то для поиска нужной записи система должна последовательно считать все предшествующие записи

Вычислить адрес нужной записи по ее номеру при такой логической организации файла невозможно, а следовательно, не может быть применен более эффективный метод прямого доступа

Файлы, доступ к записям которых осуществляется последовательно, по номерам позиций, называются **неиндексированными**, или **последовательными**, файлами

Логическая организация файла

Другим типом файлов являются **индексированные файлы**; они допускают более быстрый прямой доступ к отдельной логической записи. В индексированном файле (в) записи имеют одно или более ключевых (индексных) полей и могут адресоваться путем указания значений этих полей

Для быстрого поиска данных в индексированном файле предусматривается специальная индексная таблица, в которой значениям ключевых полей ставится в соответствие адрес внешней памяти. Этот адрес может указывать либо непосредственно на искомую запись, либо на некоторую область внешней памяти, занимаемую несколькими записями, в число которых входит искомая запись

В последнем случае говорят, что файл имеет **индексно-последовательную** организацию, так как поиск включает два этапа: прямой доступ по индексу к указанной области диска, а затем последовательный просмотр записей в указанной области. Ведение индексных таблиц берет на себя файловая система. Понятно, что записи в индексированных файлах могут иметь произвольную длину

Логическая организация файла

Все сказанное в большей степени относится к обычным файлам, которые могут быть как структурированными, так и неструктурированными. Что же касается других типов файлов, таких как каталоги или файлы типа «символьная связь», то они обладают вполне определенной специфической для своего типа структурой, которая известна и «понятна» файловой системе

Операционные системы

Лекция 15

**Функции операционной системы по
управлению файлами и устройствами**